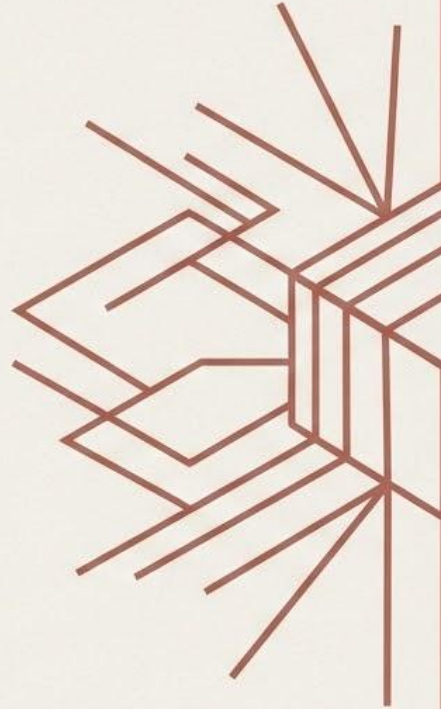


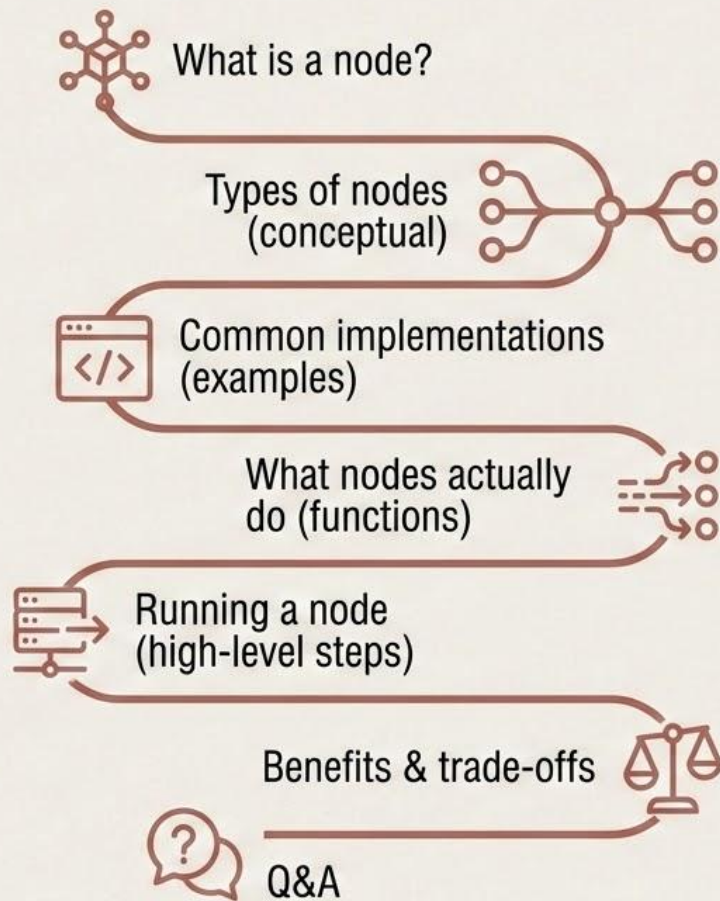
BASIC BITCOIN NODE

— What a node is, how many implementations exist, and what nodes do



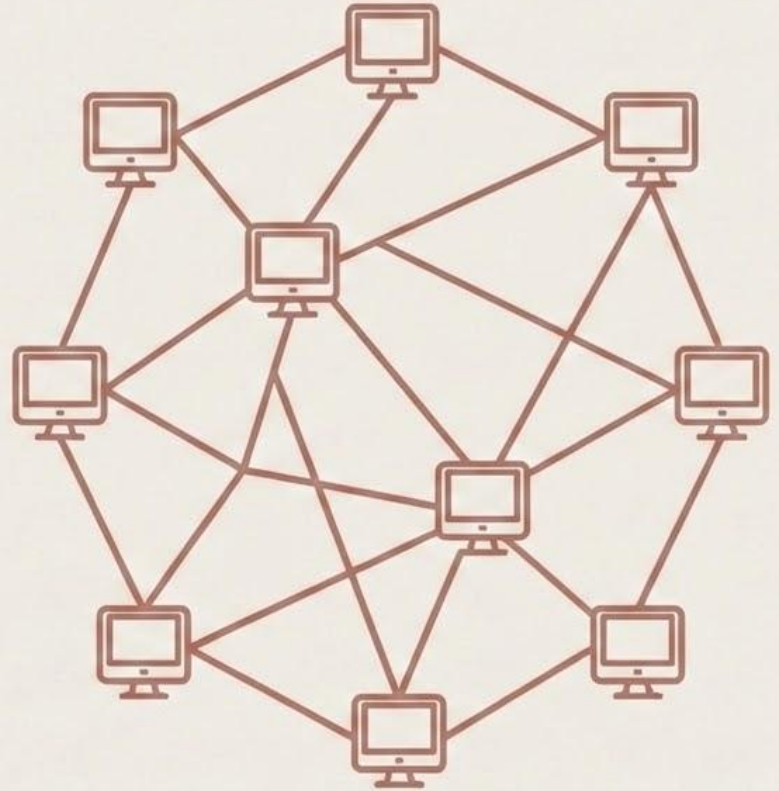
Quick roadmap

An overview of the topics we'll cover, from definition to practical considerations.



What is a node?

A computer running Bitcoin software that participates in the Bitcoin network. At the most basic level, it stores blockchain data and communicates with other nodes to exchange transactions and blocks.



Two quick clarifications



Node $\neq$ Wallet (not the same)

Wallets hold keys and create transactions; nodes help validate and relay them.



Node $\neq$ Miner (overlap exists, but different roles)

A miner might run a node, but mining is about producing blocks. Running a node is about verifying and sharing information.

Types of nodes (conceptual)



Full node — stores and validates the entire blockchain.



Pruned full node — validates but discards older block data to save disk space.



Archive node (or archival/full archival) — keeps full history and extra indexes for explorers or analytics.



SPV / Light node — verifies transactions using headers (trusts full nodes for full data).



Miner node / Mining pool node — participates in mining and block creation; usually also a full node.

Common implementations (examples)

- **Bitcoin Core** (Reference implementation, widely tested)
- **btcd** (Written in Go)
- **bcoin** (Written in Node.js)
- **Bitcoin Knots** (Alternative implementation with more features)
- **Electrum** (Light wallet using SPV model or servers)

Each project targets different languages, ecosystems, or use cases. For most users, Bitcoin Core is the default as it implements consensus rules as originally defined.

How many implementations?

- No single canonical count; open-source ecosystems evolve.
- Several widely used implementations in various languages (C++, Go, JavaScript, Rust, etc.) and many light-wallet servers.
- Diversity of implementations increases network resilience and robustness by revealing different bugs and edge cases.

What does a node do? (the core functions)

1. Validate incoming transactions and blocks against consensus rules (signatures, double-spend checks, script rules, block header difficulty, etc.).
2. Store the blockchain (full or pruned) and maintain local state (UTXO set for full nodes).
3. Relay valid transactions and blocks to peers to propagate them across the network.
4. Serve data to wallets, explorers, and other services (via RPC or P2P).
5. Protect privacy and sovereignty — you don't need to trust a third party to know the truth about the blockchain.

How nodes talk to each other (high level)

1. Peer-to-peer (no central servers)
2. Gossip protocol: new transactions and blocks are announced and requested
3. Connection limits, peer scoring, and anti-DoS measures

Running a node: basic steps (high-level, non-exhaustive)

1. Choose software (e.g., Bitcoin Core).
2. Provision hardware: sufficient disk (SSD recommended), memory, and stable internet upload.
3. Download and verify the software (check signatures).
4. Start syncing the blockchain (initial block download — IBD).
5. Configure RPC/ports and wallet access if needed.
6. Keep backups of wallet keys if you use the node's wallet.

Benefits of running a node

- Maximum trustlessness — verify consensus yourself.
- Improved privacy vs. using public nodes or third-party APIs.
- Contribute to network censorship-resistance and decentralization.
- Useful for developers and services (APIs, explorers, Lightning node backends).

Trade-offs & costs

- Storage, bandwidth, and uptime costs.
- Initial sync time.
- Slightly increased maintenance (software upgrades, monitoring).
- SPV/light wallets trade privacy and trust for convenience.

Key takeaways

- A node = a participant that validates, stores, and relays blockchain data.
- Node types: full, pruned, archive, SPV/light, miner.
- Implementations exist in multiple languages; choose based on your needs.
- Running a node gives you trustlessness, privacy, and contributes to decentralization.

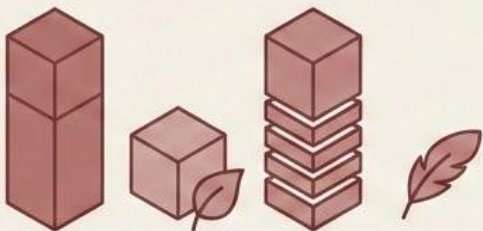
Q&A / Further resources

- I'm happy to take questions now.
- If you want next steps: I can point you to official docs, or walk through a step-by-step install on your machine.

In Short: The Bitcoin Node

A machine enforcing network rules by validating, storing, and sharing blocks.

Flavors & Trade-offs

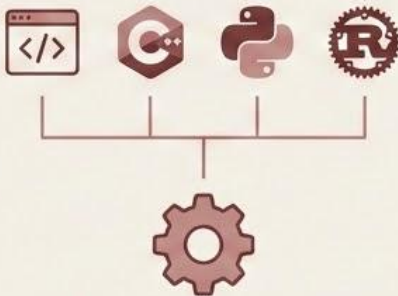


Full, Pruned, Archival, Light



Balancing resources
with sovereignty.

Implementations



Multiple languages.
Pick for your goals.

Why Run One?



Privacy &
Trustlessness



Decentralization



Bodrock